UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Block Grid Master Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Block Grid Master

Block Grid Master is a standalone editable Unity game template. Drag glass panes from the tray into empty window cells. Complete rows or columns to fire and clear them. Meet each commission goal within its pane budget; a full tray with no legal placement ends the run. Shaped windows, gems and cracked panes vary the task. Free Glazing is a local endless score mode.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Block Grid Master > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/BlockGrid/Scenes/BlockGrid_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/BlockGrid`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/blockgrid contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/BgArt.cs`
- `Scripts/BgData.cs`
- `Scripts/BgGen.cs`
- `Scripts/BgLevels.cs`
- `Scripts/BgLogic.cs`
- `Scripts/BgSession.cs`
- `Scripts/BgSfx.cs`
- `Scripts/BlockGridGame.cs`
- `Scripts/BlockGridGame.Hud.cs`
- `Scripts/BlockGridGame.View.cs`

Main flow: BlockGridGame.cs. UI and popup consumers: BlockGridGame.Hud.cs. Model/generator/loading: BgData.cs, BgGen.cs, BgLevels.cs, BgLogic.cs. Authoring: Editor/BlockGridLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Drag glass panes from the tray into empty window cells. Complete rows or columns to fire and clear them. Meet each commission goal within its pane budget; a full tray with no legal placement ends the run. Shaped windows, gems and cracked panes vary the task. Free Glazing is a local endless score mode.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact names/unlocks/free charges: Scripts/BlockGridGame.cs; effect/help strings: Scripts/BlockGridGame.Hud.cs.

Helper	Unlock	Free charges	Action and effect
Rotate	3	3	Tap Rotate, then a tray piece: it turns 90 degrees.
Undo	6	2	Takes back your last pane: board, tray and score.
Fresh Panes	10	2	Swaps the panes in your tray for a fresh set.
Sunburst	15	2	Tap Sunburst, then the window: the 3x3 area there shatters. Gems and glows count!

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Block Grid Master > Level Editor**. Exact implementation: `Editor/BlockGridLevelEditor.cs`.

Open the Block Grid Master editor and select an early level. Use Dup or Duplicate before editing. Change its visible name and one rule or board element with the exact controls below. Save explicitly and run the current entry through the editor validation and play controls. The per-game source-control table includes the current brush, geometry, target, rule and AI fields; do not treat hierarchy or resource IDs as display text.

1. Back up Resources/blockgrid/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`BlockGridLevelEditor.cs:44`	`[MenuItem("Szekipapa77/Block Grid Master/Level Editor")]`
`BlockGridLevelEditor.cs:47`	`[MenuItem("Szekipapa77/Block Grid Master/Validate All Levels")]`
`BlockGridLevelEditor.cs:108`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`BlockGridLevelEditor.cs:109`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`BlockGridLevelEditor.cs:112`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70)) && Cur != null) ValidateOne(_sel);`
`BlockGridLevelEditor.cs:113`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`

BlockGridLevelEditor.cs:116	if (GUILayout.Button(new GUIContent("Play this level", "Optional quick test: saves, opens the WebGL scene at this level and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(100)) && Cur != null) Play(_sel + 1);
BlockGridLevelEditor.cs:144	if (GUILayout.Button("Add"))
BlockGridLevelEditor.cs:150	if (GUILayout.Button("Dup") && Cur != null) { var c = Cur.Clone(); c.name += " copy"; Insert(_sel + 1, c); }
BlockGridLevelEditor.cs:151	if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete level {_sel + 1} '{Cur.name}'?", "Delete", "Cancel"))
BlockGridLevelEditor.cs:155	if (GUILayout.Button("Up") && _sel > 0) Move(-1);
BlockGridLevelEditor.cs:156	if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) Move(1);
BlockGridLevelEditor.cs:186	EditorGUILayout.LabelField($"Level {_sel + 1}", EditorStyles.boldLabel);
BlockGridLevelEditor.cs:187	l.name = EditorGUILayout.TextField("Name", l.name);
BlockGridLevelEditor.cs:188	l.hint = EditorGUILayout.TextField(new GUIContent("First-time hint", "Shown on the level card (introduce new mechanics here)"), l.hint);
BlockGridLevelEditor.cs:191	l.goal = GoalNames[EditorGUILayout.Popup("Goal", gi, GoalNames, GUILayout.Width(300))];
BlockGridLevelEditor.cs:192	if (l.Goal == GoalKind.Lines || l.Goal == GoalKind.Score) l.target = EditorGUILayout.IntField(l.Goal == GoalKind.Score ? "Target points" : "Target lines", l.target, GUILayout.Width(260));
BlockGridLevelEditor.cs:193	else EditorGUILayout.LabelField($"Target: all {l.GoalTarget} {(l.Goal == GoalKind.Glow ? "glow cells" : "gems")} on the board", GUILayout.Width(300));

`BlockGridLevelEditor.cs:196`	`l.budget = EditorGUILayout.IntSlider(new GUIContent("Pane budget", "Pieces the player may place"), l.budget, 1, 120);`
`BlockGridLevelEditor.cs:197`	`l.breather = EditorGUILayout.ToggleLeft("Breather", l.breather, GUILayout.Width(80));`
`BlockGridLevelEditor.cs:200`	`l.difficulty = EditorGUILayout.Slider(new GUIContent("Deal difficulty", "Bigger, awkward shapes as it rises"), l.difficulty, 0, 1);`
`BlockGridLevelEditor.cs:201`	`l.help = EditorGUILayout.Slider(new GUIContent("Line help", "Chance a dealt tray offers a line clear"), l.help, 0, 1);`
`BlockGridLevelEditor.cs:204`	`l.seed = EditorGUILayout.IntField(new GUIContent("Seed", "Pane colours and the random dealer"), l.seed, GUILayout.Width(240));`
`BlockGridLevelEditor.cs:205`	`if (GUILayout.Button(new GUIContent("Calibrate", "Bot-measures the budget (glow/gems) or the target (lines/score)"), GUILayout.Width(80))) { Calibrate(l, false); }`
`BlockGridLevelEditor.cs:206`	`if (GUILayout.Button(new GUIContent("Record queue", "Turns this level into a commission: records a solvable fixed piece queue from a bot run"), GUILayout.Width(100))) { Calibrate(l, true); }`
`BlockGridLevelEditor.cs:207`	`if (GUILayout.Button("Clear queue", GUILayout.Width(85))) { l.queue = ""; Changed(); }`
`BlockGridLevelEditor.cs:209`	`l.queue = EditorGUILayout.TextField(new GUIContent("Queue", "Fixed piece codes in deal order (e.g. I3.0 O2.0 L4.2). Empty = random dealer"), l.queue);`
`BlockGridLevelEditor.cs:213`	`int cols = EditorGUILayout.IntSlider("Columns", l.cols, 5, 10);`
`BlockGridLevelEditor.cs:214`	`int rows = EditorGUILayout.IntSlider("Rows", l.rows, 5, 10);`
`BlockGridLevelEditor.cs:219`	`_maskShape = BgGen.ShapeNames[EditorGUILayout.Popup("Window shape", si, BgGen.ShapeNames, GUILayout.Width(300))];`

`BlockGridLevelEditor.cs:220`	`if (GUILayout.Button("Apply shape", GUILayout.Width(90))) { ApplyMask(l, _maskShape); Changed(); }`
`BlockGridLevelEditor.cs:221`	`if (GUILayout.Button("Clear board", GUILayout.Width(90))) { for (int r = 0; r < l.rows; r++) for (int c = 0; c < l.cols; c++) if (l.At(r, c) != '#') l.Set(r, c, '.'); Changed(); }`
`BlockGridLevelEditor.cs:229`	`EditorGUILayout.LabelField("Left-click/drag paints the brush. Right-click makes the cell empty glass.", EditorStyles.miniLabel);`
`BlockGridLevelEditor.cs:231`	`EditorGUILayout.LabelField($"Gems {l.Gems} Glows {l.Glows} Cracked {l.Cracked} Frame cells {l.Voids} Panes {l.Count('p', 'g', 'k', 'O', 'K')} Queue {l.QueuePieces().Count}", EditorStyles.miniBoldLabel);`
`BlockGridLevelEditor.cs:233`	`EditorGUILayout.HelpBox("Bot: " + v + (v.Solvable ? (v.WeakRate < 0.2f ? "\nVery hard for casual players (weak bots rarely make it)." : "") : "\nNOT reliably solvable - raise the budget, lower the target or remove obstacles."),`
`BlockGridLevelEditor.cs:236`	`EditorGUILayout.HelpBox(string.Format(CultureInfo.InvariantCulture, "Last saved validation: bot {0:0}% (avg {1:0.0} panes), weak bots {2:0}%, difficulty {3:0}. Press Validate to re-run.", l.botRate * 100, l.botPieces, l.weakRate * 100, l.score), MessageType.None);`
`BlockGridLevelEditor.cs:320`	`EditorGUILayout.LabelField("Difficulty curve (validated score, else last saved score / deal difficulty; blue = breather). Click a bar to select.", EditorStyles.miniBoldLabel);`
`BlockGridLevelEditor.cs:342`	`_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (seeded, bot-validated, same BgGen code as the runtime)", true);`
`BlockGridLevelEditor.cs:345`	`_genCount = Mathf.Clamp(EditorGUILayout.IntField("Count", _genCount), 1, 100); _genStart = Mathf.Max(1, EditorGUILayout.IntField("Start at level #", _genStart));`
`BlockGridLevelEditor.cs:346`	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`

`BlockGridLevelEditor.cs:349`	`_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1); _genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);`
`BlockGridLevelEditor.cs:352`	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 8);`
`BlockGridLevelEditor.cs:353`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`BlockGridLevelEditor.cs:364`	`if (GUILayout.Button($"Generate {_genCount} level(s)", GUILayout.Height(26))) Generate();`

CHAPTER 07

Generator, insertion and replacement

Use the actual generation panel and its fields listed below. Set a reproducible seed and a small count, then review the start/index and replace option before clicking Generate. The exact generation implementation is reproduced separately with line references so append, insert and replacement behavior can be distinguished. Save and reopen the data file to verify the intended entry count; runtime generated continuation is not a project-asset save.

Source	Exact current code
`BlockGridLevelEditor.cs:42`	`bool _genReplace;`
`BlockGridLevelEditor.cs:353`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`BlockGridLevelEditor.cs:386`	`if (_genReplace) { int rm = Mathf.Min(_genCount, list.Count - at); if (rm > 0) list.RemoveRange(at, rm); }`
`BlockGridLevelEditor.cs:387`	`list.InsertRange(at, made);`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/BlockGrid/Resources/blockgrid/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "name": "First Light",
  "shape": "square",
  "hint": "Drag a pane from the tray onto the window. Fill a whole row or column to fire it!",
  "cols": 6,
  "rows": 6,
  "grid": [
    "......",
    "......",
    "......",
    "......",
    "p....p",
    "pp..pp"
  ],
  "goal": "lines",
  "target": 2,
  "budget": 12,
  "queue": "I2.0 I3.1 O2.0 I4.1 V3.0 I2.1 1.0 L4.0 I3.0 O2.0 I2.0 1.0",
  "seed": 11,
  "difficulty": 0.05,
  "help": 1,
  "breather": false,
  "botRate": 1,
  "weakRate": 1,
  "botPieces": 4,
  "score": 10
}
```


Root keys: levels. Inspect BgData.cs, BgGen.cs, BgLevels.cs, BgLogic.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/BgArt.cs`. Find `Ink`, currently `#3B2A1E`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`BgArt.cs:10 / class/static declarations`	`Amber = HzUI.Hex("#FFB547"), Copper = HzUI.Hex("#C8733A"), Cream = HzUI.Hex("#FFF4E0"), Ink = HzUI.Hex("#3B2A1E"),`
`BgArt.cs:13 / class/static declarations`	`Locked = HzUI.Hex("#4A5168"), PopupCard = HzUI.Hex("#FFF4E0"), PopupInk = HzUI.Hex("#4A3426"), BgTopC = HzUI.Hex("#2B2440"), BgBotC = HzUI.Hex("#120F1C");`
`BlockGridGame.Hud.cs:319 / HzPopup BgPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 42, PopupInk, TextAnchor.MiddleCenter, false);`

Source	Exact current code
`BgArt.cs:9 / class/static declarations`	`public static readonly Color Lead = HzUI.Hex("#1C2333"), Pewter = HzUI.Hex("#3A4458"), Slot = HzUI.Hex("#27304A"), SlotHi = HzUI.Hex("#323D5C"),`
`BgArt.cs:10 / class/static declarations`	`Amber = HzUI.Hex("#FFB547"), Copper = HzUI.Hex("#C8733A"), Cream = HzUI.Hex("#FFF4E0"), Ink = HzUI.Hex("#3B2A1E"),`
`BgArt.cs:11 / class/static declarations`	`Card = HzUI.Hex("#1E2638"), CardText = HzUI.Hex("#FFF1D6"), Sub = HzUI.Hex("#B9A98F"), Gold = HzUI.Hex("#FFD23F"),`
`BgArt.cs:12 / class/static declarations`	`Good = HzUI.Hex("#2FC27C"), Bad = HzUI.Hex("#E8455E"), Sky = HzUI.Hex("#3F6FE6"), Violet = HzUI.Hex("#9B5DE5"), Aqua = HzUI.Hex("#2EC6E8"),`

`BgArt.cs:13 / class/static declarations`	`Locked = HzUI.Hex("#4A5168"), PopupCard = HzUI.Hex("#FFF4E0"), PopupInk = HzUI.Hex("#4A3426"), BgTopC = HzUI.Hex("#2B2440"), BgBotC = HzUI.Hex("#120F1C");`
`BgArt.cs:18 / class/static declarations`	`Color.white, HzUI.Hex("#E8455E"), HzUI.Hex("#FF9A2E"), HzUI.Hex("#FFD23F"), HzUI.Hex("#2FC27C"), HzUI.Hex("#2EC6E8"), HzUI.Hex("#3F6FE6"), HzUI.Hex("#9B5DE5"),`
`BlockGridGame.cs:33 / class/static declarations`	`public static readonly string[] BoostColor = { "#E08A2E", "#3F6FE6", "#2FA56E", "#D9465E" };`
`BlockGridGame.Hud.cs:210 / void RefreshHud()`	`_budgetTxt.color = S.PiecesLeft <= 3 ? HzUI.Hex("#FF7B7B") : CardText;`
`BlockGridGame.Hud.cs:246 / void RefreshBoosters()`	`v.Ring.color = sel || _guide == b ? HzUI.Hex("#FFE066") : new Color(1, 1, 1, 0);`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/BlockGridGame.Hud.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact names/unlocks/free charges: Scripts/BlockGridGame.cs; effect/help strings: Scripts/BlockGridGame.Hud.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`BlockGridGame.cs:29 / class/static declarations`	`public static readonly string[] BoostName = { "Rotate", "Undo", "Fresh Panes", "Sunburst" };`
`BlockGridGame.cs:92 / void StartEndless()`	`BgPopup("Free Glazing", "No goal, no budget: fill the 8x8 window\nfor as long as the panes fit.\nBest score: " + Best.ToString(CultureInfo.InvariantCulture),`
`BlockGridGame.cs:127 / void ShowIntro()`	`BgPopup("Free Glazing open!", "The endless 8x8 window is open.\nTap FREE in the top bar any time\nto chase your best score.", Amber, BgArt.Get("win_badge"), null, ("Great!", Good, (Action)ShowIntro));`
`BlockGridGame.cs:137 / void ShowPreLevel()`	`var p = BgPopup("Level " + Level, msg, Amber, PreLevelArt(), D.name + (D.breather ? " - breather" : ""), ("Play", Good, (Action)Begin));`
`BlockGridGame.cs:200 / void GameOver(string why)`	`HzTween.Delay(this, 0.7f, () => BgPopup(outOfPanes ? "Out of Panes!" : "No Room Left!", msg, Bad, null, D.name, buttons.ToArray()));`
`BlockGridGame.cs:215 / void EndlessOver()`	`HzTween.Delay(this, 0.7f, () => BgPopup(record ? "New Best!" : "No Room Left!", msg, record ? Good : Bad, record ? BgArt.Get("win_badge") : null, "Free Glazing", buttons.ToArray()));`

`BlockGridGame.cs:225 / void Rescue(bool morePanes)`	`Toast($"+{BgRules.RescuePieces} panes!", Gold);`
`BlockGridGame.cs:234 / void Rescue(bool morePanes)`	`Toast("Some glass was cleared!", Gold);`
`BlockGridGame.cs:343 / void OnBooster(int b)`	`Toast(BoostName[b] + " unlocks at level " + BoostUnlock[b]); HzAudio.Play(Sfx.Invalid, 0.4f);`
`BlockGridGame.cs:349 / void OnBooster(int b)`	`if (b == BUndo && _undo == null) { Toast("Place a pane first", null, 1.2f); HzAudio.Play(Sfx.Invalid, 0.4f); return; }`
`BlockGridGame.cs:350 / void OnBooster(int b)`	`if (b == BFresh && S.Dealer.Fixed) { Toast("Commission panes can't be swapped", null, 1.4f); HzAudio.Play(Sfx.Invalid, 0.4f); return; }`
`BlockGridGame.cs:359 / void OnBooster(int b)`	`else Toast(b == BRotate ? "Tap a tray piece to turn it" : "Tap the window: the 3x3 there shatters", null, 1.4f);`
`BlockGridGame.cs:387 / void OfferCharge(int b)`	`BgPopup("+1 " + BoostName[b], "Out of " + BoostName[b] + ".\nWatch a short ad for one more?", HzUI.Hex(BoostColor[b]), BgArt.Get(BoostArt[b]), null,`
`BlockGridGame.cs:437 / void UseBurst(int r, int c)`	`if (!any) { Toast("Aim at some panes", null, 1.1f); HzAudio.Play(Sfx.Invalid, 0.4f); return; }`
`BlockGridGame.cs:546 / string PopupTitle()`	`return t ? t.GetComponent<Text>().text.Replace(' ', '_') : "popup";`
`BlockGridGame.Hud.cs:39 / void BuildHudPortrait(float W, float H)`	`_levelTxt = HzUI.Label("Level", pill.transform, "", 54, Color.white); _levelTxt.AddOutline(new Color(0.3f, 0.1f, 0, 0.35f), 3);`
`BlockGridGame.Hud.cs:41 / void BuildHudPortrait(float W, float H)`	`_nameTxt = HzUI.Label("Name", pill.transform, "", 30, new Color(1, 1, 1, 0.92f), TextAnchor.MiddleCenter, false); _nameTxt.fontStyle = FontStyle.Bold;`
`BlockGridGame.Hud.cs:47 / void BuildHudPortrait(float W, float H)`	`_budgetCard = CardAt("Budget", W / 2, y, cw, ch, Card); _budgetLbl = CardLabel(_budgetCard, "PANES", out _budgetTxt, 72);`

`BlockGridGame.Hud.cs:48 / void BuildHudPortrait(float W, float H)`	`var sc = CardAt("Score", W - 40 - cw / 2, y, cw, ch, Card); CardLabel(sc, "SCORE", out _scoreTxt, 62);`
`BlockGridGame.Hud.cs:56 / void BuildHudLandscape(float W, float H)`	`var title = HzUI.Label("Title", _layer, "Block Grid Master", 64, Amber, TextAnchor.MiddleLeft);`
`BlockGridGame.Hud.cs:59 / void BuildHudLandscape(float W, float H)`	`_levelTxt = HzUI.Label("Level", _layer, "", 46, CardText, TextAnchor.MiddleLeft);`
`BlockGridGame.Hud.cs:62 / void BuildHudLandscape(float W, float H)`	`_nameTxt = HzUI.Label("Name", _layer, "", 32, Sub, TextAnchor.MiddleLeft, false); _nameTxt.fontStyle = FontStyle.Bold;`
`BlockGridGame.Hud.cs:68 / void BuildHudLandscape(float W, float H)`	`_budgetCard = CardAt("Budget", px + cw / 2 + 10, 450, cw, 200, Card); _budgetLbl = CardLabel(_budgetCard, "PANES", out _budgetTxt, 80);`
`BlockGridGame.Hud.cs:69 / void BuildHudLandscape(float W, float H)`	`var sc = CardAt("Score", px, 645, pw, 160, Card); CardLabel(sc, "SCORE", out _scoreTxt, 62);`
`BlockGridGame.Hud.cs:86 / void BuildHudLandscape(float W, float H)`	`var t = HzUI.Label("Title", card.transform, title, 30, Sub, TextAnchor.UpperCenter, false);`
`BlockGridGame.Hud.cs:89 / void BuildHudLandscape(float W, float H)`	`value = HzUI.Label("Value", card.transform, "", valueSize, CardText);`
`BlockGridGame.Hud.cs:96 / void ComboLabel(Transform card)`	`_comboTxt = HzUI.Label("Combo", card, "", 28, Amber, TextAnchor.LowerCenter, false); _comboTxt.fontStyle = FontStyle.Bold;`
`BlockGridGame.Hud.cs:103 / void GoalCard(Image card, float w, float h)`	`_goalLbl = HzUI.Label("Title", card.transform, "GOAL", 30, Sub, TextAnchor.UpperCenter, false);`
`BlockGridGame.Hud.cs:109 / void GoalCard(Image card, float w, float h)`	`_goalTxt = HzUI.Label("Value", card.transform, "", 58, CardText);`
`BlockGridGame.Hud.cs:121 / void IconRow(float rightX, float y, float size)`	`var eb = HzUI.Button("Free", _layer, _endless ? "LEVELS" : "FREE", open ? Amber : Locked, Color.white, OnEndlessButton, 40, (int)(size * 0.3f));`

`BlockGridGame.Hud.cs:132 / void OnEndlessButton()`	`if (Level < EndlessUnlock) { Toast("Free Glazing opens at level " + EndlessUnlock); HzAudio.Play(Sfx.Invalid, 0.4f); if (_endlessBtn) HzTween.Shake(_endlessBtn, 10, 0.3f); return; }`
`BlockGridGame.Hud.cs:139 / void ConfirmLeave(string title, string msg, Action go)`	`BgPopup(title, msg, Amber, null, null, ("Yes", Good, go), ("Stay", Locked, null));`
`BlockGridGame.Hud.cs:158 / void BoosterBar(float cx, float cy, float w, float h, bool horizontal)`	`var btn = HzUI.Button("Booster" + b, parent, null, Color.white, Color.white, () => OnBooster(bb), 40, (int)(size * 0.32f));`
`BlockGridGame.Hud.cs:180 / void BoosterBar(float cx, float cy, float w, float h, bool horizontal)`	`v.BadgeText = HzUI.Label("N", bd.transform, "", (int)(size * 0.24f), Color.white); v.BadgeText.rectTransform.Stretch();`
`BlockGridGame.Hud.cs:182 / void BoosterBar(float cx, float cy, float w, float h, bool horizontal)`	`v.Caption = HzUI.Label("Cap", v.Root, "", 30, CardText);`
`BlockGridGame.Hud.cs:191 / void RefreshHud()`	`_levelTxt.text = _endless ? "FREE GLAZING" : "LEVEL " + _playLevel.ToString(CultureInfo.InvariantCulture);`
`BlockGridGame.Hud.cs:192 / void RefreshHud()`	`_nameTxt.text = _endless ? "Best " + Best.ToString(CultureInfo.InvariantCulture) : D.name + (D.breather ? " - breather" : "");`
`BlockGridGame.Hud.cs:194 / void RefreshHud()`	`_comboTxt.text = S.Streak >= 2 && S.ComboLeft > 0 ? "Combo x" + S.Streak + " (" + S.ComboLeft + ")" : "";`
`BlockGridGame.Hud.cs:197 / void RefreshHud()`	`_goalLbl.text = "BEST"; _goalTxt.text = Mathf.Max(Best, S.Score).ToString(CultureInfo.InvariantCulture); _goalTxt.color = CardText;`
`BlockGridGame.Hud.cs:198 / void RefreshHud()`	`_budgetLbl.text = "LINES"; _budgetTxt.text = S.Lines.ToString(CultureInfo.InvariantCulture); _budgetTxt.color = CardText;`
`BlockGridGame.Hud.cs:205 / void RefreshHud()`	`_goalTxt.text = S.Goal == GoalKind.Score ? prog + "\n/" + S.Target : prog + "/" + S.Target;`
`BlockGridGame.Hud.cs:208 / void RefreshHud()`	`_budgetLbl.text = "PANES LEFT";`

`BlockGridGame.Hud.cs:240 / void RefreshBoosters()`	`v.BadgeText.text = c > 0 ? c.ToString(CultureInfo.InvariantCulture) : "+";`
`BlockGridGame.Hud.cs:242 / void RefreshBoosters()`	`v.Caption.text = un ? BoostShort[b] : "Lv " + BoostUnlock[b];`
`BlockGridGame.Hud.cs:309 / HzPopup BgPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`var t = HzUI.Label("Title", ribbon.transform, title, 70, Color.white); t.rectTransform.Stretch(6);`
`BlockGridGame.Hud.cs:314 / HzPopup BgPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`var st = HzUI.Label("Sub", card.transform, subtitle, 40, Copper); st.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(cw - 80, 50), new Vector2(0, y - 25));`
`BlockGridGame.Hud.cs:319 / HzPopup BgPopupFull(string title, string message, Color accent, Sprite art, string subtitle, int stars, (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 42, PopupInk, TextAnchor.MiddleCenter, false);`
`BlockGridGame.Hud.cs:354 / void UnlockPopup(int b, Action done)`	`BgPopup("New: " + BoostName[b] + "!", BoostDesc[b] + "\nYou get " + BoostStart[b] + " free!", HzUI.Hex(BoostColor[b]), BgArt.Get(BoostArt[b]), null, ("Great!", Good, done));`
`BlockGridGame.Hud.cs:362 / void ShowSettingsPopup()`	`var p = BgPopup("Settings", "", Pewter, null, null,`
`BlockGridGame.Hud.cs:365 / void ShowSettingsPopup()`	`("How to Play", Amber, () => HelpPage(0)),`
`BlockGridGame.Hud.cs:376 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`BlockGridGame.Hud.cs:377 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`BlockGridGame.Hud.cs:391 / void HelpPage(int i)`	`BgPopup($"How to Play {i + 1}/{HelpPages.Length}", HelpPages[i], Sky, null, null,`
`BlockGridGame.Hud.cs:392 / void HelpPage(int i)`	`(last ? "Got it" : "Next", Good, last ? (Action)null : () => HelpPage(i + 1)));`

`BlockGridGame.Hud.cs:406 / void ShowTip(string text)`	`var tt = HzUI.Label("T", tip.transform, text, IsPortrait ? 32 : 36, CardText, TextAnchor.MiddleCenter, false);`
`BlockGridGame.Hud.cs:504 / void CompleteGuide(int b)`	`Toast("Nice! " + BoostName[b] + " is yours", Gold, 1.3f);`
`BlockGridGame.View.cs:301 / void RefreshPreviewRow()`	`var lbl = HzUI.Label("L", _previewRow, up.Count > 0 ? "NEXT" : "LAST PANES", 30, Sub, TextAnchor.MiddleLeft, false); lbl.fontStyle = FontStyle.Bold;`
`BlockGridGame.View.cs:498 / void FloatText(string text, Vector2 at, Color color, int size, float dur)`	`var l = HzUI.Label("Float", _fxLayer, text, Mathf.Max(28, size), color);`
`BlockGridGame.View.cs:508 / void Praise(string text, bool big = false)`	`var l = HzUI.Label("Praise", _fxLayer, text, big ? 104 : 88, Gold);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/blockgrid through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/blockgrid/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/blockgrid/Music/workshop_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.blockgrid`, version, scene orientation and signing. The build command writes `Builds/Android/blockgrid.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/blockgrid/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.